CAWASMA

Context-Aware Web & API Security Misconfiguration Analyzer

Final-year project — viva study notes

Neelam Jabeen

Frontend (Next.js web app)

Roll number: 22P-9353

Supervisor: Dr. Bilal Khan

Team: Asim Shakeel (22P-9229)
Neelam Jabeen (22P-9353)
Atta Ur Rehman (22P-9236)

These notes are generated from the repository, documentation and git history.
Every number is cited to the file or command it came from. Date: November 10, 2026.

Contents

1 The whole project **2**
1.1 The problem . . . 2
1.2 The idea . . . 2
1.3 Architecture . . . 2
1.4 Technology, and why . . . 3
1.5 Team roles and the shared contracts . . . 3

2 What Neelam built **4**
2.1 The typed API client . . . 4
2.2 Data fetching and live status . . . 4
2.3 New scan, ranked results, and explaining a score . . . 4
2.4 Live progress, history and the evaluation page . . . 5
2.5 Mock mode and the Docker/CI setup . . . 5

3 Numbers to know **5**
3.1 Tests the frontend carries . . . 6
3.2 What the UI shows . . . 6

4 Contribution timeline **6**

5 Viva preparation **6**
5.1 Likely questions, with short answers . . . 6
5.2 Three-minute demo script . . . 8
5.3 Five things to explain without notes . . . 8

6 Glossary **8**

1 The whole project

1.1 The problem

A normal security scanner gives a fixed severity to each kind of finding. A missing `X-Content-Type-Options` header is "medium" wherever it is found: on a public brochure page, and on a login API that returns credentials. The person reading the report then has to work out, by hand, which of a hundred "medium" findings actually matters. That triage is slow and easy to get wrong. Tools such as OWASP ZAP and Nuclei do find real issues, but they rate the issue, not the place it sits in.[1]

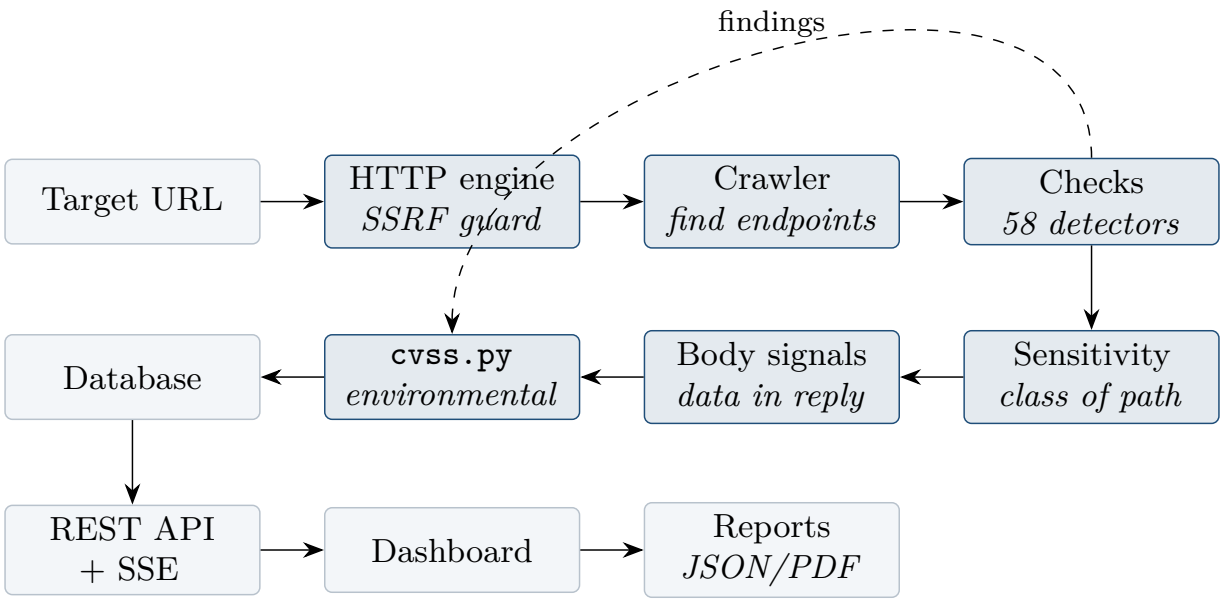
1.2 The idea

CAWASMA scores the *same* finding differently according to where it occurs and what the endpoint returned. It does this with the standard CVSS v3.1 *environmental* metrics, so the result is not an invented number: it is a recomputation of the official score with a vector anyone can check on the FIRST calculator.

One-paragraph pitch. CAWASMA is a defensive, black-box, HTTPS-only scanner for web apps and REST APIs. Each of its checks carries a fixed CVSS v3.1 base vector. For every finding it works out how sensitive the endpoint is (from the URL path) and what sensitive data the response holds (credentials, personal data, financial data, internal IPs). It turns those into CVSS environmental security requirements (CR/IR/AR) and recomputes the score, so a login API and a public page that share a weakness no longer share a priority. The point of the project is *prioritisation, not detection.*

1.3 Architecture

A scan is a pipeline. A URL goes in; a ranked, explained report comes out.



The stages are: the **engine** fetches safely (one SSRF check per redirect hop); the **crawler** discovers endpoints (seed, HTML, JavaScript, a wordlist, and OpenAPI paths); the **checks** look for misconfigurations; the **sensitivity** classifier labels each endpoint CRITICAL/HIGH/MEDIUM/LOW

[1] Source: `docs/supervisor/scanners.md`

from its path; the **body classifier** looks for sensitive data in the reply; `cvss.py` turns those two into CR/IR/AR and recomputes the score; the result is stored, served by the API over REST and Server-Sent Events, shown on the dashboard, and exported as reports.

1.4 Technology, and why

Part	Choice and reason
Scanner core	Python 3.12 with `httpx` + `asyncio`: endpoints are analysed concurrently, and the core stays database-free so it is easy to test.
Scoring	The `cvss` library, so the environmental maths is the published CVSS v3.1 formula, not ours.
NLP	Sentence-Transformers (`all-MiniLM-L6-v2`) and spaCy for the sensitivity class; the engine still works with the built-in classifier when these are absent.
API	FastAPI, with Pydantic schemas and a frozen contract; Celery + Redis run scans in the background with live status.
Database	PostgreSQL in production, SQLite in tests; the models stay portable with a `JSONType` column and database-level cascades.
TLS	SSLyze (with a plain-`ssl` fallback) for the transport checks.
Reports	One HTML template through WeasyPrint for the PDF, so HTML and PDF never drift.
Frontend	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4, shadcn/ui, TanStack Query.
Tests	`pytest` (backend), Vitest + Testing Library and Playwright (frontend).

1.5 Team roles and the shared contracts

Three people own three areas, and they meet at three written contracts, frozen so one person's change cannot silently break another's code.[2]

Member	Owns
Asim (22P-9229)	Check specifications and their CVSS v3.1 vectors, the environmental mapping, chain templates, the sensitivity and response (ML/NLP) classifiers, and the whole evaluation lab.
Neelam (22P-9353)	The Next.js web app, the `frontend` service and its CI job.
Atta (22P-9236)	The backend: HTTP engine, crawler, check functions, CVSS scoring code, attack-chain engine, reports, FastAPI app, Celery workers, database and migrations, plus the shared infrastructure and reviews.

API contract (v1). `app/api/schemas/contract.py` and `docs/api-contract.md`: the field names the frontend reads (`check_id`, `base_score`, `final_score`, status `DONE`).

ML / CVSS contract. `app/core/contracts.py` and `docs/cvss-mapping-contract.md`: the types the engine and Asim's classifiers exchange, so the engine keeps working with the built-in classifiers when the ML packages are missing.

Check-spec format. `app/core/specs.py` and `docs/check-spec-format.md`: how a check is declared so the registry can run it.

Two scoring states. Not every check is re-scored the new way yet. Of 62 check ids, 25 have an approved specification and are scored through the CVSS v3.1 environmental metrics; the other 37 still use an interim formula, $\min(10,\ \text{base} \times \text{context multiplier} + \text{response bonus})$, and each finding records which method scored it (`scoring_method`).[3]

[2] Source: `CLAUDE.md; .github/CODEOWNERS`

[3] Source: `make coverage-stats; docs/06-scoring-model.md`

2 What Neelam built

Neelam owns the whole web app in `frontend/`: a Next.js 16 application that starts a scan, shows it running live, and presents the ranked findings in a way that *explains* why each score moved. She also owns the `frontend` Docker image, its compose service and its CI job.[4]

2.1 The typed API client

What it does. `src/lib/api.ts` is the one place the app talks to the backend. Every call returns an `ApiResult` and never throws, so each screen handles success and failure the same way.

How it works. The functions are `createScan`, `getResults`, `getFindingDetail`, `cancelScan`, `listScans` and `fetchExport`. Each resolves to `{ok:true,data}` or `{ok:false,error}`, where the error is tagged `network`, `client` (4xx) or `server` (5xx). The contract types in `api-types.ts` sit over `api-schema.ts`, which is generated from `docs/openapi.json` and never hand-edited.

Listing 1: Errors are values, not exceptions (`src/lib/api.ts`).

```
export type ApiResult<T> =
  | { ok: true; data: T }
  | { ok: false; error: ApiError }; // network | client(4xx) | server(5xx)
```

Design decision. Returning results instead of throwing means no screen can forget a `try/catch`; the type forces the error case to be handled. **Inputs/outputs.** It consumes Atta's REST contract with the exact field names (`check_id`, `base_score`, `final_score`); nothing is renamed on the client. **Tested by** `src/lib/api.test.ts`. **Limitation:** the client mirrors the contract, so a contract change needs a regenerated schema (`npm run gen:api`).

2.2 Data fetching and live status

What it does. Screens read data through TanStack Query hooks, and a running scan updates itself live.

How it works. `queries.ts` has `useScanResults`, `useFindingDetail`, `useScanHistory`, `useCreateScan` and `useCancelScan`. `use-live-scan.ts` follows the Server-Sent Events stream `GET /api/scan/{id}/events`; if `EventSource` is missing or the stream drops, it falls back to polling every two seconds. When the scan reaches a terminal state the server ends the stream and the hook closes it so it does not reconnect.

Design decision. SSE first, polling as a safety net, gives live updates without a constant poll, and still works where SSE does not (the Next.js server has no `EventSource`). **Tested by** `live-status.test.ts` and the e2e flow. **Limitation:** the fallback poll adds up to two seconds of lag when SSE is unavailable.

2.3 New scan, ranked results, and explaining a score

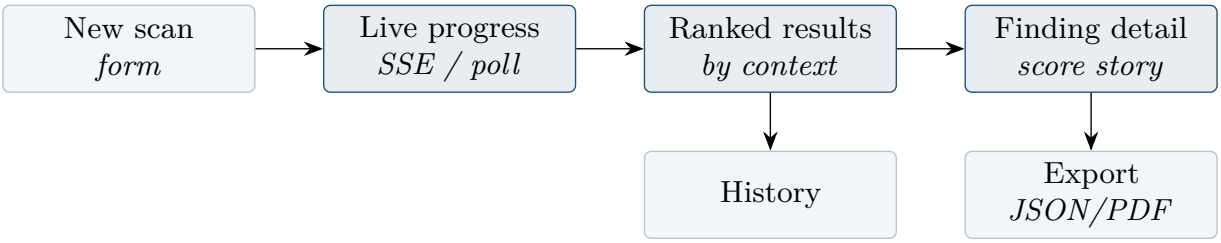
What it does. The home page starts a scan; the results page ranks the findings and, for each one, tells the story of how its score changed.

How it works.

- `scan-form.ts` validates the URL and the "I own this target" confirmation before `createScan`; the backend enforces the same rules.
- `findings-list.tsx` ranks by context-aware score, with a toggle to "Base score"; `findings-filters.tsx` filters by severity, category, OWASP and endpoint; `scan-summary.tsx` shows the severity counts and how many findings moved up or down by context.

[4] Source: `.github/CODEOWNERS; CLAUDE.md`

- `finding-detail.tsx` with `score-story.ts` shows the base score, the final score, the full CVSS vector with CR/IR/AR, the endpoint class and the body signals — the "why this score" story.



Design decision. The UI never recomputes a score; it shows the base and final numbers the backend produced and explains them, so the app and the report always agree. **Tested by** component tests and the e2e flows. **Limitation:** very large scans are paged; the UI shows up to a few hundred findings at once.

2.4 Live progress, history and the evaluation page

What it does. `live-progress.tsx` shows stages, elapsed time and counts while a scan runs; `scan-history.tsx` lists past scans with re-scan; the Evaluation page charts the project's own results.

How it works. The Evaluation page reads the final CSVs (`eval/results/final/`) through `src/data/evaluation.json`, synced by `npm run sync:eval`. Its charts are `confusion-matrix.tsx`, `ranking-compare.tsx` and `grouped-bars.tsx`. **Design decision.** The numbers are built in at build time from the evaluation output, so the page cannot drift from the harness. **Limitation:** refreshing the page's numbers needs a re-sync and rebuild.

2.5 Mock mode and the Docker/CI setup

What it does. The app runs with no backend using MSW mocks, and ships as a small standalone Docker image with its own CI job.

How it works. `src/mocks/` serves fixtures copied from `docs/examples/` (a test fails if they drift), a simulated scan that goes QUEUED → RUNNING → DONE in about ten seconds with its own SSE stream and cancel, and a 600-finding scan for performance. The `Dockerfile` builds an `output: "standalone"` server running as a non-root user; the `frontend` CI job runs lint, format, typecheck, unit tests, build and Playwright. **Design decision.** `NEXT_PUBLIC_USE_MOCK` makes every API call come from fixtures, so the UI can be developed and demoed with no backend. **Limitation:** mock mode mirrors the fixtures, so a new backend field needs the fixtures updated too.

3 Numbers to know

All figures are from the repository at commit `7cc819f`.

3.1 Tests the frontend carries

Suite	What it proves
25 unit test files (Vitest)	The API client, hooks, score story, filters, formatting.[5]
`a11y.spec.ts`	No axe accessibility violations on the main pages.
`performance.spec.ts`	The 600-finding scan renders within budget.
`definition-of-done.spec.ts`	The seven definition-of-done items pass end to end.
`eval-flow.spec.ts`, `real-stack.spec.ts`	The full flow against the real backend.
`home.spec.ts`, `demo/demo.spec.ts`	The new-scan page and the demo recording.

3.2 What the UI shows

The Evaluation page presents the project's headline numbers: detection recall (CAWASMA 0.758, Nuclei 0.765, ZAP 0.333, all precision 1.00)[6] and the base-vs-context ranking (mean NDCG@10 0.324 → 0.347).[7] The results page shows, per scan, how many findings moved up or down a severity because of context.

4 Contribution timeline

Grouped from merged pull requests.[8]

Merged	PRs	What changed
24 Oct	#6	Frontend project setup (Next.js, Tailwind, shadcn).
26 Oct	#29	Typed API client and contract types.
26 Oct	#30, #31	App shell; restore the shell lost in a merge.
26 Oct	#32	The new-scan page.
26 Oct	#33, #35	Scan results and the results view.
26 Oct	#34	End-to-end tests against the real stack.
4 Nov	#41, #42	Contract version 1.2; the finding-detail page with the score story.
4 Nov	#43	Live SSE status.
4 Nov	#44	Scan history.
4 Nov	#50, #51	Export polish; the evaluation demo page.

Reviews received. Atta pushed fixes inside `frontend/` on `atta/fix-neelam-*` branches for Neelam to review: API-schema sync (#54), fixture refreshes (#59, #64, #66), security headers (#63), responsive fixes (#73) and an unused export (#75).

5 Viva preparation

5.1 Likely questions, with short answers

1. **Server or Client Components — how do you decide?** Server Components by default; `"use client"` only where there is interactivity, such as the live scan and the filters.
2. **How does the UI show a scan running live?** `use-live-scan.ts` subscribes to the SSE stream and falls back to a two-second poll if SSE is unavailable or drops.
3. **Why both SSE and polling?** SSE gives push updates without constant requests; polling is the safety net, including on the Next.js server which has no `EventSource`.

[5] Source: `git ls-files`
[6] Source: `eval/results/final/tools.md`
[7] Source: `eval/results/final/ranking_environmental.md`
[8] Source: `gh pr list (upstream); git log`

4. **How does the client avoid crashing on an API error?** Every call returns an `ApiResult`; the type forces each screen to handle the error case, so nothing throws.
5. **How does the UI explain a score?** `finding-detail.tsx` with `score-story.ts` shows base score, final score, the CVSS vector with CR/IR/AR, the endpoint class and the body signals.
6. **Does the client ever compute a score?** No. It only displays the base and final numbers the backend produced, so the app and the PDF agree.
7. **Why generate the API types?** `api-schema.ts` comes from `docs/openapi.json` (`npm run gen:api`); the contract is the single source of truth and the client cannot drift from it.
8. **How can you develop with no backend?** `NEXT_PUBLIC_USE_MOCK` serves every call from MSW fixtures, including a simulated scan with its own SSE stream.
9. **How do you keep the mocks honest?** The fixtures are copied from `docs/examples/`, and a test fails if they drift from the backend's examples.
10. **How do you test accessibility?** `a11y.spec.ts` runs axe on the main pages and fails on any violation.
11. **How does the UI cope with 600 findings?** `performance.spec.ts` checks the 600-finding scan renders within budget; the list is filtered and paged.
12. **What is Next.js 16 specific here?** The App Router with a `src/` directory, Server Components, and APIs that differ from older Next.js, so the team checks the bundled Next docs before relying on memory.
13. **How is the app shipped?** A multi-stage `Dockerfile` with `output: "standalone"`, run as a non-root user; a separate CI job runs lint, types, unit tests, build and Playwright.
14. **How does the browser reach the API in Docker?** `NEXT_PUBLIC_API_URL` is the browser URL (build-time); `API_INTERNAL_URL` is the server-side URL (runtime, `http://api:8000`).
15. **How does the results page show the project's point?** The summary shows how many findings moved up or down a severity because of context, next to the severity counts.
16. **How does the Evaluation page get its numbers?** From `src/data/evaluation.json`, synced from `eval/results/final/` at build time, so it matches the harness.
17. **What happens when a scan is cancelled from the UI?** `useCancelScan` calls `cancelScan`; the backend sets the cancel flag and the live stream shows the scan stop.
18. **Why TanStack Query?** It caches and revalidates server state, so the screens stay in sync without hand-written fetching logic.
19. **How do you keep styling consistent?** Tailwind CSS v4 with shadcn/ui components added through its CLI.
20. **What is the biggest UI limitation?** Very large scans are paged, and the Evaluation page needs a re-sync and rebuild to update its numbers.
21. **How do you stop the API types drifting from the backend?** They are generated from `docs/openapi.json`, and a test checks the fixtures against the backend's examples, so a mismatch fails CI.
22. **What does the finding-detail page let the user verify?** It links to the FIRST CVSS calculator for both the base and the environmental score, and shows the OWASP and CWE standards for the check.
23. **Why can the SSE mock only run in some places?** `EventSource` exists in the browser and in tests but not on the Next.js server, so the mock's `sse()` is registered only where it exists.
24. **How is CORS handled between the app and the API?** The API's allow-list (`FRONTEND_ORIGIN`) defaults to the app's origin (`localhost:3000`); if the app is served elsewhere, both change together.
25. **Why keep the lockfile npm-10 compatible?** Node 22 ships npm 10; installing with npm 11 must be followed by `npx npm@10 install`, or `npm ci` fails in Docker and CI.

26. **How does the results page make the project's point obvious?** The summary shows how many findings moved up or down a severity because of context, right next to the plain severity counts.
27. **What runs in the frontend CI job?** Lint, format check, typecheck, unit tests, the production build, and Playwright — plus a separate job that runs the full flow against the real backend.

5.2 Three-minute demo script

1. Open the home page. Fill a target, tick "I own this target", start a scan.
2. Watch the live progress. Say: "This is the SSE stream; it falls back to a two-second poll."
3. Open the ranked results. Toggle "Base score" and back. Say: "Context re-ranks the list; the base order is one click away."
4. Open a finding. Walk through base → final, the CVSS vector, the endpoint class and the body signals.
5. Export the PDF, then open the History and Evaluation pages.

5.3 Five things to explain without notes

1. The `ApiResult` pattern: errors are values, every screen handles them.
2. Live status: SSE first, two-second poll as a fallback.
3. The finding-detail score story: base, final, vector, class, signals.
4. The app never recomputes a score; it displays and explains the backend's.
5. Mock mode and how the fixtures are kept honest.

6 Glossary

CVSS base score
The standard severity of a vulnerability on its own, 0–10, from a vector such as `AV:N/AC:L/...`. Fixed per check; never invented per scan.

CVSS environmental score
The base score recomputed for a particular environment, using CR/IR/AR and modified base metrics. CAWASMA's context-aware score is this.

CR / IR / AR
Confidentiality, Integrity and Availability *Requirements*: environmental inputs saying how much each property matters here. Values Low, Medium, High, or X (Not Defined). CAWASMA sets them from the endpoint class and the data in the response.

Sensitivity class
CAWASMA's label for how sensitive an endpoint is — CRITICAL, HIGH, MEDIUM or LOW — from the words in its URL path.

Body signals
Sensitive data found in a response: credentials, personal data (PII), financial data, or an internal IP address. Evidence is redacted.

Base vs. final score
CAWASMA always keeps both: the official base score and the context-aware (environmental) final score, in separate columns.

Legacy vs. environmental scoring
A finding is scored either by the CVSS environmental metrics (approved specs) or by the interim legacy formula; its `scoring_method` says which.

Precision
Of the findings that were judged, the fraction that were correct: $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$.

Recall
Of the real issues in the answer key, the fraction found: $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$.

FPR
False-positive rate: $\mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$.

Ground truth
A hand-written list, per target, of which check should (`present`) or should not (`absent`) fire on which endpoint.

NDCG@10
Normalised Discounted Cumulative Gain at rank 10: a score for how well a ranking puts the important items near the top, against a graded list.

Tie-break sampling
Base scores tie often, so a ranking's NDCG is averaged over 1000 random orderings of the tied items, with a fixed seed.

SSRF
Server-Side Request Forgery: tricking the scanner into requesting an internal address. CAWASMA guards against it by validating the resolved IP on every hop.

Soft-404
A page that returns HTTP 200 but is really "not found". The crawler learns a baseline so it does not treat these as real endpoints.

Passive vs. active scan
Passive sends only safe GET/HEAD and reads what comes back; active sends extra, still non-destructive, probes. CAWASMA never sends attack payloads.

BOLA / IDOR
Broken Object-Level Authorisation / Insecure Direct Object Reference: reading another user's object by changing an id. CAWASMA only flags an *indicator*; confirming it needs logged-in accounts, which is future work.

Attack chain
Several findings that together make a bigger risk, correlated from a template and scored with their own composite vector.

Environmental metric "X"
"Not Defined": the neutral baseline. A public (LOW) endpoint uses X, so its score is unchanged — context only ever raises a score here, it does not lower a public page below its base.